Wrapper types and autoboxing

Objectives

- Identify where autoboxing and unboxing happen when primitive values go into and come out of a collection
- Predict the result of comparing two Integer objects with == and with equals, and of unboxing a null Integer

Teaching

Start with a guess

A list of numbers holds `Integer` objects. Read this code before any explanation.

```
List<Integer> ids = new ArrayList<>();
ids.add(1000);
ids.add(1000);
System.out.println(ids.get(0) == ids.get(1));
```

Collections hold objects, so Java converts

A type argument must be a reference type: `List<int>` does not compile. Each primitive has a wrapper class, such as `Integer` for `int` and `Double` for `double`. The compiler converts for you:

- Autoboxing, primitive to wrapper: `ids.add(5)` passes an `Integer`.
- Unboxing, wrapper to primitive: `int first = ids.get(0);` and arithmetic such as `ids.get(0) + 1`.

== compares objects; equals compares values

Between two `Integer` objects, == asks "the same object?" and `equals` asks "the same value?". Java guarantees the same object only for boxed values from -128 to 127; for others, such as 1000, == may be `false`. Compare wrappers with `equals`, or unbox first.

Unboxing `null` throws `NullPointerException`. `Map.get` returns `null` for a missing key, so `int n = counts.get("x");` throws when `"x"` is absent.

Apply it to a stock map

A shop keeps a `Map<String, Integer>` of items left in stock.

Model

The model: a collection holds references to wrapper objects, never raw `int` values. Going in, an `int` is boxed into an `Integer`; coming out, it is unboxed back. Two boxes can hold the same value and still be different objects, so == on wrappers asks about objects and `equals` asks about values. An empty reference, `null`, has no value to unbox.

Recap

- Collections store objects, so `int` values are autoboxed to `Integer` going in and unboxed coming out or in arithmetic.
- == on two `Integer` objects compares identity; it is guaranteed `true` for equal values only from -128 to 127. Use `equals` for values.
- Unboxing `null` throws `NullPointerException`, which often comes from `Map.get` on a missing key.

Next topic: Big-O basics

Cheat sheet

Boxing	`list.add(5)`, `Integer n = 5;`
Unboxing	`int n = list.get(0);`, `list.get(0) + 1`
Value comparison	`a.equals(b)`
Safe map read	`int n = counts.getOrDefault("x", 0);`

Optional example. Try the step 1 prediction before you read on: this example explains its answer.

Picture each `Integer` as a small box with a number card inside. A list holds only boxes, so Java puts the number in a box on the way in (boxing) and takes the card out on the way back (unboxing).

Two boxes can hold cards with the same number. `equals` opens both and compares the cards; == only asks whether you are holding the very same box. For values from -128 to 127, Java guarantees that boxing hands out the same box each time, so == agrees with `equals`; for 1000 you may get two different boxes. And `null` is no box at all, so there is no card to take out: unboxing it throws `NullPointerException`.

Sources

Claims on this page were checked against the live official pages below on 13 October 2026 (UTC), for Java SE 25. Program output was confirmed on OpenJDK 25.0.4 with default settings. Links open outside this lesson.

- Dev.java: Autoboxing and Unboxing: when the compiler boxes and unboxes.
- Dev.java: Numbers: the wrapper classes.
- Dev.java: Restrictions on Generics: cannot instantiate generic types with primitive types.
- Integer (Java SE 25 API): valueOf always caches -128 to 127 and may cache others.
- Java Language Specification SE 25, chapter 5: 5.1.7 Boxing Conversion; 5.1.8 Unboxing Conversion.

Implementation detail, not a Java promise: == printing `false` for 1000 is what OpenJDK 25 does with default settings; the specification allows either result.

Static interaction alternatives

predict

Prediction (practice, not scored): a List<Integer> gets add(1000) twice; what does ids.get(0) == ids.get(1) print? Options: true, both are 1000 (wrong: == compares the Integer objects, not their values); false on a standard JDK, and Java promises neither result (correct: boxing 1000 twice may create two objects; OpenJDK 25 prints false); a compile error (wrong: == on two Integer objects compiles); NullPointerException (wrong: both elements exist).

drag

Sorting activity, ids is a List<Integer>: ids.add(5) autoboxes the int 5. int first = ids.get(0) unboxes. Integer total = 0 autoboxes. total = total + ids.get(1) unboxes both sides, adds, then boxes the result: both. String s = ids.get(0).toString() is neither: it calls a method on the Integer object.

explore

Comparison bench: Integer a and b are both boxed from the same value. For 100 or 127, a == b prints true (guaranteed for -128 to 127) and a.equals(b) prints true. For 128 or 1000, a == b printed false on OpenJDK 25 with default settings (not guaranteed either way) while a.equals(b) prints true. Unboxing first, a.intValue() == b.intValue(), compares values and prints true.

mcq

Apply question: stock is a Map<String, Integer> with no key pear; int left = stock.get("pear") throws NullPointerException, because get returns null and unboxing null fails. Wrong: left is 0; a compile error; left is null. Retry form: Integer discount = null; if (discount == 0) also throws NullPointerException, because comparing with the int 0 unboxes discount.